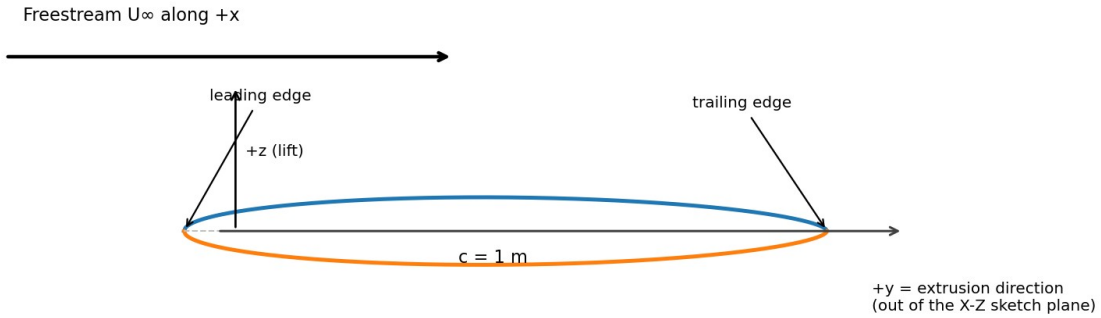
FROM AIRFOIL DATA TO CFD EVIDENCE

A start-to-current-milestone learning guide for the cusped-leading-edge airfoil in OpenFOAM Foundation 13

Geometry • Meshing • Compressible flow • Surface data • Forces • Verification • Validation



Coordinate convention used in this case (schematic, not the exact STL profile).

What this document is

A tutorial from the beginning of the project through the work completed so far, with the reasoning behind each operation and a continuation plan. It is written so a new reader can understand the case without needing the previous chat or the two earlier PDFs.

Case: ~/Projects/cusped-airfoil/openfoam-case | OpenFOAM Foundation 13 | Solver module: shockFluid

Status: 2 November 2026. The custom mesh and startup simulation exist; volume and surface post-processing, wall-stress analysis, force integration and independent force cross-checks have been completed for the saved final snapshot. Physical validation remains open.

How to use this guide

Read it in order the first time. Each stage answers a different engineering question. Do not jump straight to force coefficients: those numbers depend on correct geometry, mesh, flow variables, surface data and reference quantities.

Every lesson follows the same pattern:

- Question: what are we trying to find out?
- Physics: what does the quantity mean?
- OpenFOAM operation: what is calculated, read or written?
- Procedure: commands and files, with the reason for each step.
- Interpretation: what the current case actually showed.
- Limitations: what must be checked before treating the result as trustworthy.

Reading map

Part	What it teaches
Part I	Understand the research question, case structure and physical model
Part II	Build the airfoil geometry, computational mesh and solver case
Part III	Run the transient case and audit numerical health
Part IV	Post-process the volume: fields, statistics, VTK and line sampling
Part V	Move from near-wall information to actual wall-surface data
Part VI	Wall pressure, probes, wall shear, geometry checks and forces
Part VII	Validate, extend the simulation and report reproducibly

Important distinction

A successful run, a plausible-looking contour, a numerically consistent force integral and a physically validated CFD result are four different levels of evidence. This project has reached the third level for its current snapshot, not the fourth.

Project status at a glance

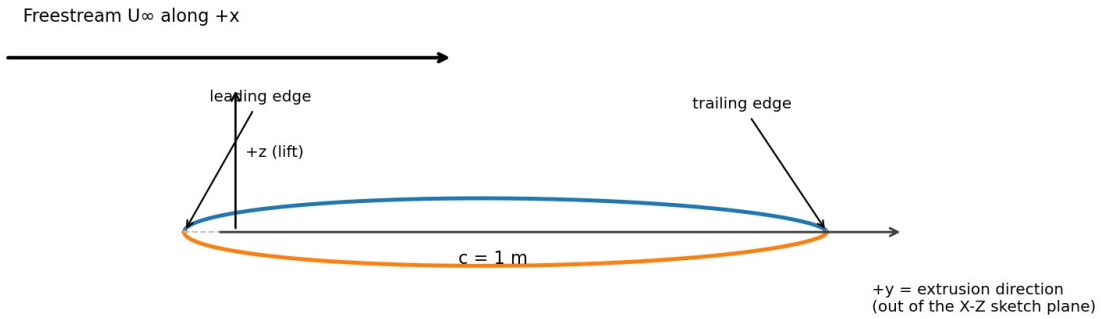
Area	Current state	Interpretation
Geometry	Custom cusped profile, STL checked and transformed	Usable starting geometry; preserve coordinate source and mesh provenance.
Mesh	24,000 hexahedral cells; `Mesh OK`; max non-orthogonality 87.27°	Topology passes, but high non-orthogonality is a serious quantitative-quality warning.
Flow setup	Laminar, compressible ideal gas, Sutherland viscosity; `shockFluid`	Tutorial-adapted startup model; pressure convention and exact paper correspondence still require review.
Run	0 to 2e-5 s; four saved transient snapshots	A startup demonstration, much shorter than one chord-flow time.
Post-processing	Ma, |U|, vorticity, line sample, wall patch, probes, shear and forces	Substantial extraction and analysis completed on saved fields.
Force cross-check	Python integration and Foundation v13 force objects agree	Strong verification of current force integration and coefficient normalization.
Validation	Mesh/domain/time-step sensitivity not completed	Do not present the current coefficients as final aerodynamic performance.

PART I - Understand the problem before opening OpenFOAM

1. The physical problem and what the paper is studying

The project is based on Akram and Hasan, “The dynamics of supersonic flow past a new cusped leading edge airfoil” (2024). The paper studies unsteady, viscous, laminar, compressible flow around a symmetric cusped-leading-edge airfoil and compares it with a conventional biconvex airfoil. The reported parameter space includes Reynolds number 5×10^5, Mach numbers from 1.25 to 2.13 and angles of attack 0°, 10° and 20°. The exact setup and geometry must be taken from the paper itself before claiming a reproduction.

The project case uses one initial operating point: nominal Mach 1.25, zero angle of attack, chord c=1 m, T∞=300 K, p field set to 1775.544 Pa and U∞=(433.9,0,0) m/s. The pressure value needs a convention check because the paper text identifies it as gauge pressure while an ideal-gas equation of state needs absolute thermodynamic pressure.



The profile is in the X-Z plane. The span/extrusion coordinate is Y.

Why compressible flow?

At Mach 1.25, density changes cannot be ignored. Pressure, temperature and density are coupled through the energy equation and the equation of state. An incompressible solver would not represent the intended shock/expansion physics. `shockFluid` is the selected Foundation v13 solver module for this compressible case.

Why laminar?

The reference paper describes a laminar flow model. Therefore the local `constant/momentumTransport` selects `simulationType laminar` rather than inheriting the diffuser tutorial’s turbulence model. This is a modelling choice based on the reference study, not a claim that every real flow at the same Reynolds number is laminar.

What is the actual research goal?

The first goal was modest: make the geometry, mesh and solver work and learn how to interrogate the result. The later goal is to reproduce and then extend the paper. A startup run does not satisfy the later goal. The guide deliberately keeps those milestones separate.

2. Essential CFD quantities and the numbers used here

The case is built around air modelled as a perfect gas. The basic relationships below let you check the input state independently of OpenFOAM.

Quantity	Relationship	Current reference value
Specific gas constant	$R = Ru / W$	≈287.0 J/(kg K)
Speed of sound	$a\infty = \sqrt{(\gamma R T\infty)}$	≈347.2 m/s
Freestream speed	$U\infty = M\infty\ a\infty$	≈433.9 m/s for M=1.25
Density	$\rho\infty = p\infty/(R T\infty)$	≈0.02062 kg/m³ if $p\infty$ is absolute
Dynamic pressure	$q\infty = 0.5\ \rho\infty\ U\infty^2$	≈1940.95 Pa

Quantity	Relationship	Current reference value
Chord Reynolds number	Re = ρ∞ U∞ c / μ∞	≈4.85×10^5 using the recorded state and Sutherland μ
Chord convective time	tc = c/U∞	≈2.304 ms

The case molecular weight is 28.966 kg/kmol; Cv=719.4 J/(kg K). For a calorically perfect gas Cp=Cv+R and γ=Cp/Cv, giving γ≈1.40. The viscosity model uses Sutherland constants As=1.458×10^-6 and Ts=110.4 K.

Pressure convention must be resolved

Do not use a gauge pressure as absolute pressure in ρ=p/(RT). The recorded density 0.02061885 kg/m³ follows from treating 1775.544 Pa as absolute pressure at 300 K. If that value is gauge pressure, the thermodynamic absolute pressure and density differ. This affects Reynolds number, dynamic pressure, Cp and force coefficients.

3. How an OpenFOAM case is organised

3.1 Prerequisites and environment check

Before creating or reopening the case, have the Foundation v13 installation, the paper-derived coordinate file, the STL conversion script, the two local reference tutorials, and the case dictionaries available. The current machine uses the following installation path; another system may use a different one.

```
source /opt/OpenFOAM/OpenFOAM-13/etc/bashrc
foamVersion
echo "$FOAM_TUTORIALS"
command -v foamRun
test -d "$FOAM_TUTORIALS/fluid/aerofoilNACA0012" && echo "mesh tutorial found"
test -d "$FOAM_TUTORIALS/shockFluid/diffuserIntake" && echo "solver tutorial found"
python3 -m pip show numpy matplotlib
```

The check confirms that the shell is using Foundation 13, the solver executable is on PATH, both reference tutorials are visible, and the Python plotting packages are present in the interpreter you intend to use. Python packages are optional for the solver itself; they are used later for analysis and plotting.

OpenFOAM is dictionary- and directory-driven. A solver reads the dictionaries and fields, advances the equations and writes time folders. Post-processing reads those saved fields and either calculates extra fields or extracts data. It does not advance physical time unless you run the solver again.

```
openfoam-case/
├── 0/                    initial fields and boundary conditions
│   ├── p                  pressure field
│   ├── T                  temperature field
│   └── U                  velocity vector field
├── constant/
│   ├── polyMesh/           generated computational mesh
│   ├── triSurface/         STL surfaces used by meshing
│   ├── geometry/           transformed/scaled geometry assets
│   ├── physicalProperties    thermophysical model and constants
│   └── momentumTransport     laminar/turbulence model selection
├── system/
│   ├── blockMeshDict        multi-block mesh definition
│   ├── extrudeMeshDict      one-layer extrusion definition
│   ├── controlDict          solver module, time and output controls
│   ├── fvSchemes            spatial/temporal discretisation schemes
│   ├── fvSolution           linear solver and algorithm controls
│   ├── graphUniform         line-sampling configuration
│   ├── forceCoeffsCompressible  coefficient function-object settings
│   ├── forcesCompressible    dimensional force settings
│   └── functions            function-object inclusion list
├── 5e-06/, 1e-05/, ...      saved solution snapshots
├── postProcessing/          tables and sampling products
├── VTK/                   VTK conversion products
└── log.foamRun              captured solver log
```

The three main locations are `0`, `constant` and `system`. The time folders are data, not configuration. `postProcessing` and `VTK` contain analysis outputs, not new CFD solutions.

Two different OpenFOAM tutorial sources were combined

- `$FOAM_TUTORIALS/fluid/aerofoilNACA0012`: used as a structural example for a multi-block airfoil mesh and the blockMesh → transformPoints → extrudeMesh workflow.
- `$FOAM_TUTORIALS/shockFluid/diffuserIntake`: used as a starting point for `shockFluid`, thermophysical properties and numerical settings. Its turbulence model was not retained; the local case was changed to laminar.

Tutorials are references, not truth

Copying a tutorial creates a starting point. Every geometry, boundary, physical model and numerical setting must be checked against the new problem.

PART II - From coordinate data to a running case

4. Geometry: coordinate data, STL and coordinate conventions

An airfoil `.dat` file normally contains ordered profile coordinates. The geometry is not just a picture: the coordinate order, units, closure at the trailing edge, orientation and extrusion direction determine what OpenFOAM sees. The project used the paper-derived coordinate file `cusped_le_akram_hasan.dat` and a Python conversion script (`dat_to_stl.py`) to create the original STL. The exact triangulation implementation should be preserved alongside that script; the guide does not invent its undocumented internals.

4.1 Establish the axes and scale

- x is the chordwise direction from leading edge x=0 to trailing edge x=1 m.
- z is the airfoil thickness/lift direction; upper and lower surfaces lie at positive and negative z.
- y is the span/extrusion direction. The 2D calculation is represented by one thin cell layer in y, with front and back set to `empty`.

The reported profile has c=1 m, maximum thickness close to 0.05c, and the maximum-thickness location near x/c=0.75. The current STL bounds after the first conversion were x=0..1, y≈-0.0489542..0.0489542 and z=0..0.1 m. That means the source STL had been made in a different plane from the final CFD convention.

4.2 Check the original STL before transforming it

```
surfaceCheck constant/triSurface/cusped_airfoil.stl
```

`surfaceCheck` checks triangulation, connected regions, open/closed edges, invalid triangles and bounding box. In this project it reported 2,388 triangles, 1,196 vertices, one connected part and a closed surface, with no illegal triangles. Very small triangles/close point pairs near the sharp feature were also reported; those are worth remembering because a cusp is geometrically demanding.

4.3 Transform the profile into the mesh coordinate system

```
surfaceTransformPoints "Rx=90, translate=(0 0.05 0)" \
  constant/triSurface/cusped_airfoil.stl \
  constant/triSurface/cusped_airfoil_xz.stl

surfaceCheck constant/triSurface/cusped_airfoil_xz.stl
```

The 90-degree rotation moves the profile from the XY sketch plane into XZ. The translation recentres the original 0..0.1 extrusion thickness around y=0, yielding y≈-0.05..+0.05. Always check the resulting bounding box rather than trusting a transform command by sight.

4.4 Make a spanwise-scaled projection surface

```
surfaceTransformPoints "scale=(1 10 1)" \
  constant/triSurface/cusped_airfoil_xz.stl \
  constant/geometry/cusped_airfoil_span1.stl

surfaceCheck constant/geometry/cusped_airfoil_span1.stl
```

This scaled copy spans y=-0.5..+0.5 m. It is a construction surface used by the tutorial-derived projected mesh workflow; it is not the actual final 0.1 m computational span. The final mesh is collapsed/converted and extruded to a thin y extent. Keep the original STL, transformed STL and construction STL as separate files so the transformations remain auditable.

Geometry checkpoint

Before meshing, confirm: chord direction is +x; profile is in XZ; maximum thickness and its x-location match the intended coordinates; the STL is closed; there is one connected body; and all dimensions are in metres.

5. Constructing the mesh: why a multi-block grid?

The mesh is the numerical representation of the fluid domain. The governing equations are integrated over cells and fluxes are passed across faces. Cell orientation and size affect how accurately gradients, shocks and wall shear can be represented. A structured multi-block mesh gives control over cell alignment and grading around the airfoil, particularly where a boundary layer and compressible disturbances are expected.

The case adapted the Foundation `aerofoilNACA0012` blockMesh strategy: several hexahedral blocks form the surrounding domain; selected edges are projected onto the transformed STL; the spanwise coordinate is collapsed for construction and the 2D plane is then extruded to a one-cell-thick 3D mesh with empty front/back patches.

5.1 Mesh pipeline and what each utility does

Command	What it does	Why it is used
`blockMesh`	Creates the multi-block hexahedral grid described by `system/blockMeshDict`.	Defines cell topology and approximate cell locations around the profile.
`transformPoints "scale=(1 0 1)"`	Collapses the y coordinate onto the X-Z plane.	Converts the spanwise construction mesh to the 2D profile plane.
`extrudeMesh`	Extrudes the plane in y to a thin one-layer volume.	Creates a valid OpenFOAM mesh with front/back `empty` patches for 2D equations.
`checkMesh`	Checks topology and cell/face geometry.	Finds invalid topology and warns about distortion before solving.

```
blockMesh 2>&1 | tee log.blockMesh
transformPoints "scale=(1 0 1)" 2>&1 | tee log.transformPoints
extrudeMesh 2>&1 | tee log.extrudeMesh
checkMesh 2>&1 | tee log.checkMesh
```

`tee` writes the terminal output into a log while still displaying it. In Bash, use `set -o pipefail` so a failed OpenFOAM command is not hidden by the successful `tee` process.

5.2 What is inside blockMeshDict?

The dictionary defines vertices, blocks, cell counts, grading, edges, boundary patches and projections onto the STL. The case uses six hexahedral blocks. The important controls in the recorded final stage were:

Setting	Recorded value	Meaning
xUCells	80	Number of cells in the upstream block; increased from 30 to 60 to 80.
xMCells	30	Cells in the middle block.
xDCells	40	Cells in the downstream block.
zCells	80	Vertical/radial block resolution.
xUGrading	2	Cell-size grading in upstream block.
leadGrading	0.2	Grading control near leading edge.
xDGrading	10	Downstream grading.
zGrading	40	Vertical grading.

The dictionary contained intended horizontal controls xMin=-4 and xMax=4, but the actual mesh bounding box was x=-1.25..4 because the upstream curved boundary projected onto a circle centred at x=0.75 with radius 2 m. The actual generated mesh, not a variable name, is authoritative. That discrepancy is an explicit reminder to check the mesh bounds after generation.

5.3 What did checkMesh tell us?

Mesh iteration	Cells	Max skewness	Skewness faces	Max non-orthogonality	Faces >70°
Initial	16,000	8.715	12	84.73°	60
Upstream 60; grading 2	20,800	4.213	1	86.76°	138
Upstream 80; grading 2	24,000	3.082	0	87.27°	186

The upstream refinement removed the skewness failures so the final run printed `Mesh OK`. But non-orthogonality remained very high and the number of faces above 70° increased. Skewness and non-orthogonality are different metrics: improving one does not guarantee improvement in the other. `Mesh OK` means the checks selected by the utility passed; it is not a certificate of gradient or wall-stress accuracy.

The final bounding box was x=-1.25..4 m, y=-0.1..0 m, z=-2..2 m. The grid had 24,000 hexahedral cells, 48,680 points and 96,340 faces. There are five patches: `aerofoil`, `inlet`, `outlet`, `front`, `back`. The front/back patches are `empty`; the airfoil boundary has 220 faces.

```
checkMesh -writeSurfaces -surfaceFormat vtk
find postProcessing/checkMesh -type f -print
```

This writes diagnostic surfaces for mesh-quality regions. A flagged face should be mapped to coordinates and a location before changing the mesh. In this case poor-quality faces clustered around the leading-edge region; this is where the sharp cusp and block topology interact.

Mesh checkpoint

The final mesh is usable for learning the workflow and inspecting preliminary flow structure. It is not yet suitable evidence for final wall gradients, shock position or aerodynamic coefficients without mesh-quality repair and sensitivity testing.

6. 2D modelling by one thin extrusion layer

The airfoil profile is two-dimensional in X-Z. OpenFOAM uses a 3D mesh data structure, so this case is represented by one thin cell layer in y. The front and back faces are assigned `empty`, telling OpenFOAM that there is no variation or flux in that coordinate and the problem is two-dimensional in X-Z.

The final mesh extent y=-0.1..0 m means the cell-centre plane of the single uniform layer is y=-0.05 m. This matters for graph sampling and point probes. One earlier note recommended y=0 for sampling; that is not the centre of the recorded final mesh. Use the generated mesh coordinates and cell centres, not a copied coordinate from an older note. For this recorded mesh, y=-0.05 m is the natural mid-plane.

7. Building the compressible laminar solver setup

The flow solver configuration was adapted from `shockFluid/diffuserIntake`. The selected module is `shockFluid`, run through `foamRun`. The local model is set to laminar. The main governing conservation laws are mass, momentum and energy; compressibility means density and temperature evolve with pressure and velocity.

7.1 Thermophysical model

The case uses a perfect-gas equation of state, sensible internal energy, constant Cv and Sutherland temperature-dependent dynamic viscosity. The configuration recorded in `constant/physicalProperties` is:

```
thermoType
{
    type            hePsiThermo;
    mixture         pureMixture;
    transport       sutherland;
    thermo          eConst;
    equationOfState perfectGas;
    specie          specie;
    energy          sensibleInternalEnergy;
}

// Recorded case properties (representative entries)
molWeight       28.966;
Cv              719.4;
As              1.458e-06;
Ts              110.4;
```

The snippet shows the model choices and principal values, not a substitute for the complete dictionary syntax in the live case. `hePsiThermo` is a compressible thermodynamics package; `perfectGas` relates pressure, density and temperature; `sutherland` makes viscosity depend on temperature; `eConst` supplies constant specific heat; sensible internal energy is the energy variable used by this configuration.

7.2 Laminar model

```
simulationType laminar;
```

The project set `simulationType laminar` in `constant/momentumTransport`. The `shockFluid` run log then selected the laminar stress model `Stokes` and the laminar thermophysical transport model. Do not accidentally keep the diffuser tutorial's turbulence model.

7.3 Initial fields and boundary conditions

The `0` folder defines p, T and U. The initial/reference state is pInf=1775.544 Pa, TInf=300 K and UInf=(433.9 0 0) m/s for alpha=0°. The physical intent of the boundaries is shown below; always inspect the actual `0/p`, `0/T`, `0/U` files because patch settings are field-specific.

Patch	Pressure p	Temperature T	Velocity U	Physical intent
inlet	fixedValue pInf	fixedValue TInf	fixedValue UInf	Impose undisturbed incoming state.
outlet	zeroGradient	zeroGradient	zeroGradient	Allow solution to leave without fixing each field value.
aerofoil	zeroGradient	zeroGradient	noSlip	No-slip solid wall; zero normal-gradient p and adiabatic-style T treatment.
front/back	empty constraint	empty constraint	empty constraint	Suppress variation across the thin direction for a 2D X-Z calculation.

The wall `T` zeroGradient condition corresponds to zero normal temperature gradient, commonly used as an adiabatic-wall representation. It does not mean the wall temperature must remain at 300 K; the wall temperature can respond to the adjacent compressible flow.

Model assumptions to keep visible

The pressure convention, exact Reynolds number, Prandtl/thermal transport representation, outer boundary formulation and numerical schemes must be checked against the paper before claiming a faithful reproduction. The current setup is an adapted first case.

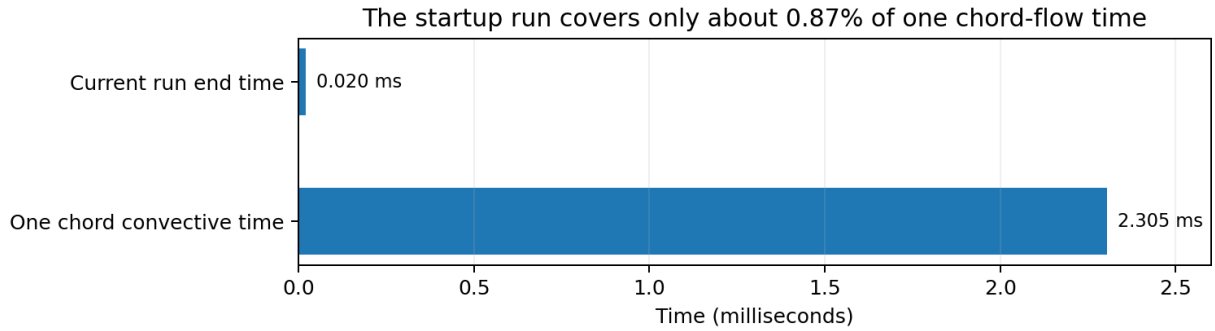
8. Run controls and the first solver run

The current run used deltaT=1e-7 s, endTime=2e-5 s and writeInterval=5e-6 s. That is 200 time steps from 0 to 20 microseconds, with a saved result every 50 steps.

```
set -o pipefail
foamRun 2>&1 | tee log.foamRun
```

`foamRun` advances the discretised flow equations in physical time. At each write interval it stores p, T, U and other registered fields in a time directory. The case wrote: `5e-06`, `1e-05`, `1.5e-05` and `2e-05`. These are four snapshots from one simulation, not four independent simulations.

The run reached `End` at 2e-5 s. At the last step, mean Courant number was about 0.001952 and maximum Courant number about 0.484876. Representative final linear residuals were very small. This demonstrates that the solver advanced stably for this short interval; residuals are algebraic-solver measures, not direct physical-error estimates.



The recorded 20 μs startup interval is only a small fraction of one chord convective time.

One chord convective time is $c/U_\infty = 1/433.9 \approx 2.30$ ms. The simulated interval is 0.020 ms, approximately 0.87% of that time. Therefore the solution is an early startup snapshot, not a settled aerodynamic state or a time-averaged result.

PART III - Begin post-processing: understand the saved solution

9. Reopen the case safely

Use this at the start of a working session. Environment setup and case navigation are separate from post-processing.

```
source /opt/OpenFOAM/OpenFOAM-13/etc/bashrc
cd ~/Projects/cusped-airfoil/openfoam-case
foamListTimes
ls -lh 2e-05/
tail -n 30 log.foamRun
```

If the OpenFOAM installation path differs on another machine, source that installation's own `etc/bashrc`. Do not source one release and use executables from another. `foamListTimes` shows saved time directories. `ls` confirms which fields exist at the selected time.

9.1 Keep the original run safe

```
mkdir -p notes logs plots scripts reports
cp system/controlDict system/controlDict.bak-$(date +%Y%m%d-%H%M%S)
cp system/graphUniform system/graphUniform.bak-$(date +%Y%m%d-%H%M%S)
```

Create backups before editing dictionaries. Keep the original STL, `.dat` coordinates, Python conversion script, mesh dictionaries, physical properties, boundary fields and solver log. Never casually run `foamCleanCase` or `foamListTimes -rm`; these can remove mesh or result data.

10. Solver run versus post-processing

Operation	What it does	Advances CFD time?
`foamRun`	Solves the flow equations using the current fields and dictionaries.	Yes
`foamPostProcess ... -func MachNo`	Calculates Mach number from saved thermodynamic and velocity fields.	No
`foamPostProcess ... -func vorticity`	Calculates curl of velocity from saved U.	No
`foamPostProcess ... -func graphUniform`	Samples fields at locations along a line.	No
`foamToVTK -latestTime`	Converts saved mesh/fields to VTK representation.	No
Python plotting	Reads exported values and creates plots/statistics.	No

This distinction matters. If the user asks for more physical time, extend `foamRun`. If the user asks for more measurements of an existing time, use `foamPostProcess`, VTK or Python. Post-processing cannot make the flow evolve.

11. Check numerical health before interpreting physics

```
tail -n 50 log.foamRun
grep -E 'Time =|Courant Number|Solving for|ExecutionTime|FOAM FATAL|^End' log.foamRun
foamListTimes
checkMesh
```

Read the log in this order:

- `End` means the solver reached its configured stopping time without a fatal exit.
- Courant number estimates how far information travels relative to cell size in one time step. A low maximum value supports time-step stability but is not an accuracy certificate.
- Initial and final residuals describe the linear system solution for each equation at a time step. Small final residuals do not prove steady flow or correct boundary conditions.
- Field minima/maxima can reveal unexpected values, but a global min/max does not say where the value occurs.
- Mesh warnings remain relevant even after the solver runs successfully.

For this case, maximum Co was about 0.485, the solver reached 2e-5 s, and no fatal exit was reported. However, the mesh retains max non-orthogonality 87.27° and 186 faces above 70°. This is especially important for gradients and wall quantities.

12. Compute derived volume fields

The saved solution has primitive fields such as p, T and U. Many engineering questions require quantities calculated from those fields. A derived field is a new measurement computed from the saved result; it is not a new solve.

12.1 Discover commands supported by this exact installation

```
foamPostProcess -list
foamPostProcess -help
foamPostProcess -list | grep -i -E 'MachNo|vorticity|wallShearStress|graphUniform|forces|patchSurface'
```

Foundation and OpenCFD/OpenFOAM.com distributions can differ in function-object syntax and options. The installed Foundation v13 utility and local packaged templates are authoritative for this case. The official v13 guide lists field calculations, force functions, graph sampling, probes and surface functions.

12.2 Mach number

Mach number is local flow speed divided by the local speed of sound: $M=|U|/a$. For a perfect gas, $a=\sqrt{(\gamma p/\rho)}$. It accounts for local changes in thermodynamic state; velocity magnitude alone does not.

```
foamPostProcess -solver shockFluid -latestTime -func MachNo 2>&1 | tee log.postMach
ls -lh 2e-05/Ma
```

The command creates the scalar field `Ma` at the latest saved time. The `-solver shockFluid` option provides solver context needed by the compressible thermophysical model. The result should be close to 1.25 in undisturbed upstream flow; locally, acceleration/compression/expansion changes it.

12.3 Speed magnitude

```
foamPostProcess -solver shockFluid -latestTime -func "mag(U)" 2>&1 | tee log.postMagU
ls -lh '2e-05/mag(U)'
```

U is a vector with x, y and z components. `mag(U)` produces $|U|=\sqrt{(Ux^2+Uy^2+Uz^2)}$, a scalar speed in m/s. It answers “how fast?” but not “in which direction?”.

12.4 Vorticity

```
foamPostProcess -solver shockFluid -latestTime -func vorticity 2>&1 | tee log.postVorticity
ls -lh 2e-05/vorticity
```

Vorticity is $\nabla\times U$ and has units s^-1. It highlights rotational/shear regions, but its magnitude is built from spatial velocity gradients. It is therefore particularly sensitive to cell geometry, resolution and numerical interpolation. A very large peak (an earlier estimate was around 3.5×10^6 s^-1) must be mapped to its coordinates before interpretation.

12.5 Check the field statistics, not just the first lines of a field file

OpenFOAM stores internal cell values in mesh order. The first few values printed by `head` are not automatically leading-edge values or freestream values. A robust analysis parses all values, checks that the number read equals the declared count, then computes statistics.

```
head -n 30 2e-05/Ma
head -n 30 2e-05/p
```

Recorded global scalar-field statistics at 2e-5 s:

Field	Minimum	Maximum	Mean	Median
p (Pa)	977.077	2183.41	1776.4	1775.54
T (K)	255.148	358.499	300.091	300
Ma (-)	0.302456	1.36951	1.24904	1.25011

The mean and median Mach values being near 1.25 are consistent with most cells remaining near the freestream value. The minimum and maximum show that local flow regions differ substantially. But global extrema do not identify where those cells are. That leads to the next step: mapping fields to physical coordinates.

12.6 Map cell values to cell-centre coordinates

```
foamPostProcess -solver shockFluid -latestTime -func writeCellCentres
ls -lh 2e-05/C*
```

Cell-centre coordinates can be combined with p, T, Ma or vorticity values. This makes it possible to ask where a minimum or maximum occurs, rather than only reporting the value. Check actual filenames written by this Foundation v13 function in the time folder.

13. Export volume and wall fields for spatial inspection

A numerical field is a list of values associated with mesh cells or faces. A contour is a spatial view of those values. It helps answer where a feature is located, but visual appearance alone does not identify its cause.

```
foamToVTK -latestTime -ascii
find VTK -maxdepth 3 -type f | sort
```

The command converts latest-time volume and patch data into VTK files. The recorded volume file was `VTK/openfoam-case_200.vtk`; the airfoil patch file was `VTK/aerofoil/aerofoil_200.vtk`. The patch export contains 220 wall cells and 440 points, and its cell data included `Ma`, `rho`, `p`, `T`, `U`, `wallShearStress` and `vorticity`. The volume file contains all 24,000 volume cells.

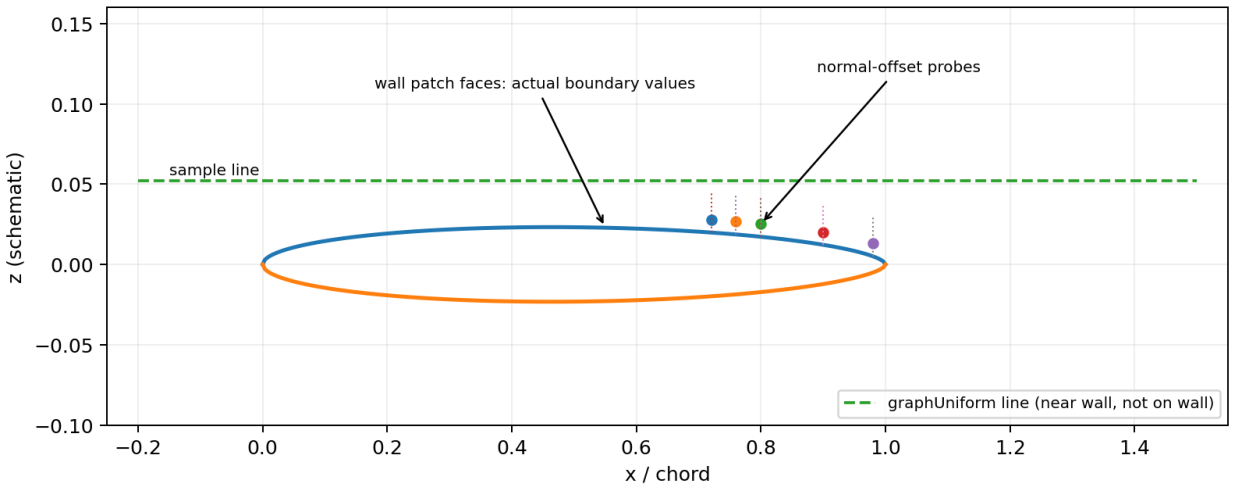
Use a spatial viewer such as ParaView when the engineering question is spatial: locate a pressure rise, inspect an expansion region, see the shock topology, or examine the boundary layer. For each figure, record the field, time, units, colour range, plane/patch and what it does not prove.

Visualisation discipline

Use the field contour to locate a feature. Then extract coordinates and values with sampling or scripts. Do not call a feature a shock based only on a colour transition; a shock interpretation requires consistent pressure, temperature, Mach and spatial structure, with sufficient mesh resolution.

14. Line sampling: a diagnostic path through the fluid

A line sample produces a table of field values along a specified path. It is useful for inspecting pressure, temperature, Mach or velocity variation with position and for measuring how quickly a disturbance changes. But the line only tells you what happens along that chosen path.



A near-wall line is in the fluid. Wall-face values are obtained from the actual wall patch. Normal-offset probes sample selected fluid points.

14.1 Why choose a line near the upper surface?

The old line was placed at z=0.052 m, 3.05 mm above the maximum airfoil height of approximately 0.048954 m near x=0.75. It is therefore a near-wall fluid line, not the actual solid wall. The earlier work sampled x from -0.2 to 1.5 m using 200 uniform points and fields p, T and U.

The single y-cell layer spans y=-0.1 to 0 m; its centre plane is y=-0.05 m. The older guide included an inconsistent note recommending y=0 and calling y=-0.05 an empty plane. The recorded mesh bounds imply the opposite: y=-0.05 is the centre of this one-layer extrusion. Recheck cell centres if the mesh is regenerated or changed.

14.2 Inspect the actual graphUniform dictionary

```
cat system/graphUniform
```

The case uses the Foundation v13 packaged graph function. In principle the important entries are `start`, `end`, `nPoints`, `fields` and `axis`. The exact include syntax is version/template dependent, so keep the local dictionary and packaged v13 template as the source of truth.

```
// Example geometric settings for the near-upper-surface line
start   (-0.2 -0.05 0.052);
end     (1.5 -0.05 0.052);
nPoints 200;
fields  (p T U);
axis    x;
```

The values above express the actual geometry and variables; use the rest of the locally valid dictionary structure and `#includeEtc` configuration already present in the case.

14.3 Run the sample and inspect the header

```
foamPostProcess -solver shockFluid -latestTime -func graphUniform 2>&1 | tee logs/graph-near.log
find postProcessing/graphUniform -type f -print | sort
head -n 8 postProcessing/graphUniform/2e-05/line.xy
```

The actual output header was `x p T U_x U_y U_z`. Mach was requested in one configuration attempt but did not appear in the output. Never assume a field made it into a sample: the header tells you exactly which columns exist. Mach can be reconstructed from sampled velocity and local temperature for a perfect gas.

14.4 Plotting the line data

Python is separate from OpenFOAM. A missing `numpy` error usually means the interpreter used to run the script is not the same interpreter where NumPy was installed. Check `which python`, `python -m pip show numpy` and `python -c "import numpy"`. A dedicated virtual environment makes the plotting environment repeatable.

```
python3 -m venv ~/.venvs/openfoam-post
source ~/.venvs/openfoam-post/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy matplotlib
python -c "import numpy, matplotlib; print(numpy.__version__, matplotlib.__version__)"
```

The plotting script reads the numeric columns, computes speed, density-independent local sound speed from T and γR, derives Ma=|U|/√(γRT), and plots p, T, Ma and Uz versus x. The line plot showed changes near x≈0.65-0.8 m; Mach peaked around 1.31 and dipped toward roughly 1.24 in the plotted region. This local line evidence helped motivate surface-focused analysis, but it did not provide wall properties.

Why we changed the question

A line 3.05 mm above the body is useful for near-wall flow structure, but it is not the airfoil surface. Once the question became “what pressure, temperature and shear act on the airfoil?”, we needed to extract the `aerofoil` wall patch itself.

PART IV - Move from the fluid field to the airfoil surface

15. Why surface extraction is a different operation

The volume fields p, T and U are stored at cell centres. A solid wall is represented by boundary faces (the `aerofoil` patch). A surface analysis must therefore work with wall-face data or a controlled interpolation to those faces. Sampling a nearby fluid cell and calling it wall pressure silently changes the measurement location.

There are three distinct data questions:

Method	Location	Question answered
Volume field	All fluid cells	What is happening through the computational domain?
Line sample	A defined path in the fluid	How does a variable change along this path?
Wall patch	Faces belonging to `aerofoil`	What pressure, temperature or shear is associated with the solid boundary?
Point probes	Chosen points in the fluid	How do selected local values vary in space or time?

16. Export the airfoil wall patch

The actual wall patch name is `aerofoil`, verified in `constant/polyMesh/boundary`. OpenFOAM's VTK conversion can export both volume data and patch data.

```
foamToVTK -latestTime -ascii
find VTK/aerofoil -maxdepth 1 -type f -print
```

The project produced `VTK/aerofoil/aerofoil_200.vtk`. It contains wall geometry and both cell and point data. This file is the preferred one for extracting values attached to wall faces in this case.

16.1 Understand cell data versus point data

A VTK patch mesh has points (vertices) and cells (here wall faces). A face-centred pressure or shear value is naturally associated with a wall face. Point data are interpolated/averaged to vertices and can be convenient for smooth visualisation, but they are not identical to original face values. For integrated forces or face-by-face tables, use cell data when available.

The earlier `postProcessing/surfaces/2e-05/aerofoilWall.vtk` contained geometry plus point fields only. It can be useful for visualisation but is not the preferred source for this face-based analysis. The `foamToVTK` patch file included cell and point fields.

16.2 Verify the patch content before processing

Inspect the VTK file in a reader (for example PyVista) and list: number of points, number of cells, cell-data arrays and point-data arrays. The recorded patch file had 440 points and 220 cells. Cell arrays included `patchID`, `mag(U)`, `Ma`, `rho`, `p`, `T`, `wallShearStress`, `U` and `vorticity`. This is consistent with the mesh having 220 faces on the airfoil boundary.

Surface data checkpoint

Before plotting: confirm the file is the airfoil patch, confirm the selected array is cell data or point data, confirm the physical time is 2e-5 s, and retain the face-centre coordinates with each value.

17. Read and interpret wall pressure and temperature

Pressure on the wall is the local static pressure at the airfoil boundary. Temperature at the wall reflects the local compressible-flow state and the adiabatic wall treatment; it is not forced to remain at the freestream temperature. The surface plot can show loading, expansions, compression features and local heating/cooling.

The recorded face-centre data were approximately symmetric between upper and lower sides, as expected for the symmetric geometry and zero-angle-of-attack case. Representative face locations included:

x/c (approx.)	z/c upper/lower	p (Pa)	T (K)	|τw| (Pa)
0.9133	±0.02082	1125.75	265.60	5.909

x/c (approx.)	z/c upper/lower	p (Pa)	T (K)	|τw| (Pa)
0.7878	±0.04370	1300.05	276.07	6.336
0.7406	±0.04891	1729.99	297.91	7.334
0.4971	±0.03601	2142.60	317.73	7.885
0.3185	±0.01899	2169.23	319.02	8.472
0.1867	±0.00760	2058.66	314.52	9.551
0.0893	±0.00192	1917.19	309.45	13.089
0.0176	±0.000081	1805.09	328.58	39.566

These values describe one saved startup snapshot and selected face centres. They are not a converged Cp curve. The near-leading-edge shear rises sharply; because the mesh is highly non-orthogonal and the geometry has a sharp cusp, that peak requires particular caution.

17.1 A sharp surface jump near x/c≈0.754

Surface data showed an abrupt change around x/c=0.754202: pressure changed from about 1870.44 Pa to 1345.60 Pa (a drop of roughly 524.84 Pa) and temperature from about 304.727 K to 278.551 K. The geometry slope also changes rapidly in this region. This could be associated with a real local expansion/flow feature, but it could also be affected by discrete surface resolution, geometric faceting or mesh quality. It should not be labelled a shock based on this jump alone; a shock normally involves a compression/pressure rise and must be inspected in the surrounding volume field.

18. Pressure coefficient Cp: why the reference state matters

Cp converts dimensional surface pressure into a nondimensional pressure loading using the freestream dynamic pressure:

$Cp = (p_{wall} - p_\infty) / q_\infty, \quad$ where $q_\infty = 0.5\, \rho_\infty\, U_\infty^2$

With the recorded values treated as absolute: $\rho_\infty \approx 0.02061885$ kg/m³, $U_\infty = 433.9$ m/s and $q_\infty \approx 1940.9477$ Pa. But the paper's pressure convention remains unresolved. Until that is settled, any computed Cp curve should be labelled provisional and its reference pressure/density declared.

What is needed for an actual surface Cp curve?

- Read wall pressure from the face-centred `aerofoil` patch data, not from the near-wall graph line.
- Keep each face centre x coordinate; calculate x/c with c=1 m.
- Use one consistent p_∞, ρ_∞ and U_∞, with the absolute/gauge convention checked.
- Calculate Cp face-by-face and sort by x/c separately for upper and lower surfaces.
- State whether the plotted Cp axis is inverted; many airfoil plots invert it for visual convention.
- Repeat on refined meshes and compare the Cp curve before interpreting small features.

Status of Cp

The wall pressure field has been extracted and analysed. A final paper-comparable Cp distribution is still conditional on resolving the freestream pressure convention and demonstrating mesh/time sensitivity.

19. Near-wall probing: quantify how the disturbance decays away from the wall

A wall patch says what is happening at the boundary. A normal-offset probe says how the surrounding fluid changes as you move away from it. This distinction is important for boundary-layer and expansion-region analysis.

The project placed probes at x/c stations 0.72, 0.74, 0.76, 0.80, 0.85, 0.90 and 0.98, using normal offsets of 0.002, 0.005, 0.010, 0.020, 0.040, 0.080 and 0.120 m. This produced 49 probe locations. A robust script used a mesh cell locator (`find_containing_cell`) and confirmed all 49 points were inside valid cells; this corrected an earlier script that returned NaNs because it had failed to find the containing cells.

x/c	Offset (m)	Ma	p (Pa)	T (K)	Ux (m/s)	Uz (m/s)
0.74	0.002	1.2789	1870.44	304.727	447.28	+9.13
0.76	0.002	1.2449	1333.52	277.979	414.45	-35.16
0.76	0.005	-	1553.22	-	-	-
0.76	0.010	-	1752.84	-	-	-
0.80	0.002	1.2411	1287.77	275.369	410.76	-40.10
0.90	0.002	1.2205	1147.92	266.993	396.27	-51.86
0.98	0.002	1.1917	1040.23	260.030	380.30	-60.51

At x/c=0.74, the near-wall flow at 2 mm offset is locally faster and higher in pressure/temperature than the freestream; at x/c=0.76 and aft, the near-wall sample shows lower pressure and temperature, reduced streamwise speed and increasingly negative z-velocity. At several positions the disturbance moves back toward freestream by around 0.02 m normal offset. These are local observations, not a standalone proof of a shock, expansion fan or separation bubble.

Why use an actual cell locator? A point sampler must identify which mesh cell contains the requested coordinate. If the point lies outside the mesh, inside the solid, on an invalid/empty boundary or outside a supported interpolation domain, the script may return NaNs. Always count successful point-to-cell mappings and inspect failed coordinates before interpreting data.

PART V - Wall shear, surface geometry and integrated loads

20. Wall shear stress

A no-slip wall creates a velocity gradient between zero wall velocity and the moving fluid. Viscous stress is related to that gradient. `wallShearStress` is a vector wall field; for a Newtonian fluid it represents the tangential viscous stress on the wall as implemented by the solver's stress model.

```
foamPostProcess -solver shockFluid -latestTime -func wallShearStress
ls -lh 2e-05/wallShearStress
```

The exported VTK wall patch included `wallShearStress` as a three-component vector. Its dimensions in the OpenFOAM field were [1 -1 -2 0 0 0 0], equivalent to kg/(m s^2) or Pa.

Recorded wall-shear magnitude: minimum ≈5.085 Pa; maximum ≈92.458 Pa. Upper-side maximum ≈92.380 Pa; lower-side maximum ≈92.458 Pa. The global maximum was near x≈0.00188 m, very close to the leading edge. The peak is highly sensitive to near-wall resolution and mesh quality; it should not be reported as a reliable physical peak until those checks are done.

20.1 Skin-friction coefficient

A common magnitude-based definition is $C_{f,mag} = |\tau_w|/q_\infty$. Using $q_\infty \approx 1940.9477$ Pa gives the preliminary range:

Quantity	Value
Minimum $\|\tau_w\|$	5.084997 Pa
Maximum $\|\tau_w\|$	92.457946 Pa
Minimum $\|\tau_w\|/q_\infty$	0.00261985
Maximum $\|\tau_w\|/q_\infty$	0.04763547

This was a magnitude-based local coefficient. A signed conventional Cf curve requires projecting the stress onto the local surface tangent and establishing a consistent sign convention. In the analysis, the raw tangential projection was negative on all faces because of the vector direction convention. Do not silently plot a magnitude as if it were a signed skin-friction coefficient.

21. Check wall-face normals and surface areas

A force calculation depends on the wall-face area vectors. Each face has an area magnitude and a normal direction. If the face normals are wrong or inconsistent, pressure-force directions can be wrong even when the pressure values themselves look reasonable.

The analysis separated the 220 wall faces into 110 upper and 110 lower faces. Mean normals were approximately upper (-0.008839, 0, -0.99308) and lower (-0.008839, 0, +0.99308). Summing the oriented area vectors over the closed body gave a vector close to zero: approximately (8.44×10^-11, 0, 4.41×10^-17). This is a useful closure check for a closed surface.

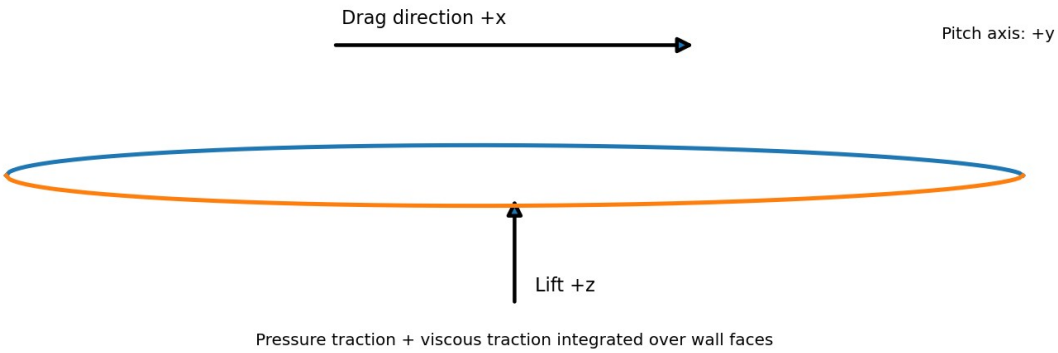
Surface measure	Value
Upper wall area	0.10068413 m^2
Lower wall area	0.10068413 m^2
Total wetted wall area	0.20136826 m^2
Chosen reference area Aref	0.10000000 m^2
Chord c	1.0 m
Extrusion thickness	0.1 m

The wetted wall area is not the same as the reference area used in the aerodynamic coefficient. The selected Aref is chord × extrusion depth = 1.0×0.1=0.1 m^2. The upper and lower surface areas sum to about 0.20137 m^2 because both sides are wetted. Do not substitute total wetted area for planform/reference area.

22. Integrate pressure and viscous forces

A surface plot describes local loading. An integrated force answers the engineering question: what net force does the flow exert on the airfoil? Conceptually, the total wall load is the integral of pressure traction and viscous

traction over the airfoil boundary. The force object handles the exact face-normal and stress calculations consistently with the OpenFOAM solver.



Force directions for the project's coordinate system (schematic).

For this model, x is the flow/drag direction, z is the lift direction and y is the pitch axis. `CofR=(0.25 -0.05 0)` defines the centre about which moments are calculated. The reference chord is 1 m and reference area is 0.1 m².

22.1 Dimensional forces with forcesCompressible

The Foundation v13 `forcesCompressible` function object reports pressure, viscous and (if present) porous forces as dimensional vectors, plus moments about the chosen centre of rotation. The case-specific dictionary created was:

```
/* OpenFOAM Foundation v13 */
patches       (aerofoil);
CofR          (0.25 -0.05 0);
pitchAxis     (0 1 0);
log           yes;
#includeEtc "caseDicts/functions/forces/forces.cfg"
```

It is saved as `system/forcesCompressible`. The `#includeEtc` line imports the Foundation v13 packaged settings rather than manually reconstructing implementation details. This is safer than copying a template from another OpenFOAM distribution.

22.2 Nondimensional coefficients with forceCoeffsCompressible

`forceCoeffsCompressible` normalises dimensional loads using dynamic pressure and reference geometry. The configured dictionary was:

```
patches       (aerofoil);
magUInf       433.9;
rhoInf        0.02061885;
lRef          1.0;
Aref          0.1;
CofR          (0.25 -0.05 0);
dragDir       (1 0 0);
liftDir       (0 0 1);
pitchAxis     (0 1 0);
pRef          1775.544;
log           yes;
#includeEtc "caseDicts/functions/forces/forceCoeffs.cfg"
```

The case-specific file is `system/forceCoeffsCompressible`. `pRef` is supplied consistently with the case's pressure setup; its appropriateness still depends on resolving the absolute/gauge convention. The coefficient formulae are:

$C_d = D / (q_\infty A_{ref})$ | $C_l = L / (q_\infty A_{ref})$ | $C_m = M_y / (q_\infty A_{ref} c_{ref})$

Here $q_\infty \approx 1940.9477$ Pa, $A_{ref}=0.1$ m² and $c_{ref}=1$ m, so $q_\infty A_{ref} \approx 194.0948$ N. This is why a dimensional drag of 11.0883 N corresponds to $C_d \approx 0.05713$.

22.3 Run both functions together

A `system/functions` dictionary included both function objects:

```
FoamFile
{
    format      ascii;
    class       dictionary;
    location    "system";
    object      functions;
}
#includeFunc forceCoeffsCompressible
#includeFunc forcesCompressible
```

```
foamPostProcess -dict system/functions -solver shockFluid -latestTime
```

This runs post-processing on the saved latest time. It does not advance the flow. The console reported both objects, then wrote coefficients and dimensional forces.

22.4 Actual force results at t=2e-5 s

Contribution	Fx (N)	Fz (N)	Meaning
Pressure	+9.178529	-0.000146746	Pressure loading integrated over the airfoil wall.
Viscous	+1.909740	+0.000015988	Viscous wall traction integrated over the airfoil wall.
Total	+11.088269	-0.000130758	Net load; positive x is the configured drag direction.

Moment contribution about +y	My (N m)
Pressure	-3.640959×10^-5
Viscous	+3.967361×10^-6
Total	-3.244223×10^-5

Coefficient	Value at 2e-5 s
Cd	0.057128 approximately
Cl	-6.73682×10^-7
Cm	-1.67146×10^-7
Cl(f)	-5.03987×10^-7
Cl(r)	-1.69695×10^-7

Pressure contributes about 82.8% of the total x-force and viscous traction about 17.2%. The near-zero lift is consistent with a symmetric geometry at zero angle of attack, though this is a symmetry/consistency observation, not proof of a converged aerodynamic solution. `Cl(f)` and `Cl(r)` in this output are front/rear contributions associated with moment reporting; they are not upper- and lower-surface lift values.

23. Independently cross-check the force integration

A force value can be numerically consistent within one implementation while still being wrong because of a wrong reference area, wrong pressure convention, wrong normal, wrong patch or wrong direction. The Python calculation provided a second independent route: it integrated the face-centred pressure and viscous wall data using extracted face areas/normals, then applied the same reference normalisation.

Quantity	Python integration	OpenFOAM function object
Pressure drag	9.178528612 N	9.178529 N
Viscous drag	1.909739899 N	1.909740 N
Total drag	11.088268512 N	11.08827 N
Total lift	-0.000130760 N	-0.000130758 N
Cd	0.057128115	0.057128 approximately

The difference is only at displayed rounding precision. This agreement is strong evidence that the Python face areas, integration directions and coefficient normalisation match OpenFOAM's own force integration for this snapshot.

What the cross-check proves - and what it does not

It verifies the calculation path for the current mesh, wall patch, saved fields, reference state and snapshot. It does not prove the mesh is accurate, the boundary conditions are appropriate, the solution has settled, or the result matches the paper.

PART VI - From a verified snapshot to a trustworthy CFD result

24. The status of each result

Result	What we have established	What remains open
Mach field	Ma was calculated and freestream statistics are close to 1.25.	Spatial interpretation and mesh/time sensitivity of local extrema.
Line profile	A repeatable table/plot showed disturbance near x≈0.65-0.8.	Line is near-wall fluid data, not a wall pressure curve; verify sampling plane after mesh changes.
Wall pressure and temperature	Actual aerofoil patch data were extracted and mapped.	Resolve pressure reference and test mesh/time sensitivity before final Cp.
Near-wall flow	49 probe positions were valid; local p/T/U/Ma trends were quantified.	Refine near-wall mesh; check interpolation and repeat at additional times.
Wall shear	Vector field and magnitude/Cf,mag were extracted.	Leading-edge peak and signed tangential Cf need improved mesh and sign convention.
Surface geometry	Upper/lower areas and normal closure were checked.	Repeat after geometry or mesh modifications.
Forces	Dimensional pressure/viscous loads and Cd/Cl/Cm were computed.	Time history, mesh/domain/time-step independence and paper comparison.
Independent verification	Python and OpenFOAM force results agree closely.	This is code/data consistency, not physical validation.

25. Next step: determine whether forces are changing with time

The current force values were computed at only the latest time, 2e-5 s. The solver wrote four snapshots. The immediate next post-processing exercise is to calculate the same force quantities at each saved time and compare them. A single value cannot tell whether the flow is still developing.

```
for t in 5e-06 1e-05 1.5e-05 2e-05; do
  echo "===== time $t ====="
  foamPostProcess -dict system/functions -solver shockFluid -time "$t"
done
```

After running, inspect the coefficient and force data files:

```
find postProcessing -type f \
 \( -iname '*forces*' -o -iname '*forceCoeffs*' \) -print | sort
find postProcessing/forceCoeffsCompressible -type f -name '*.dat' -print | sort
find postProcessing/forcesCompressible -type f -name '*.dat' -print | sort
```

The exact output directory naming can depend on function-object execution. Confirm the files and time stamps rather than assuming a file has been written. Plot Cd(t), Cl(t), Cm(t), and dimensional pressure/viscous forces. With only 20 microseconds total, these are likely early transient trends, not a statistically meaningful time history.

26. Add point probes for time histories

A line sample answers “how does the field vary with position at one time?” A probe answers “how does a field at one physical point evolve with time?” Probes complement the integrated force history by showing local pressure, temperature, velocity or Mach response.

```
foamGet probes
cat system/probes
```

Inspect the Foundation v13 template and configure fields plus fixed physical locations. Place probes deliberately: upstream freestream, near the leading edge, near the region around x/c≈0.75, and downstream. Ensure each point lies in a fluid cell, not inside the solid or outside the domain. The resulting time history should be checked for monotonic drift, periodic behaviour, boundary reflections or local spikes.

27. Extend the run only when you intend to simulate more physical time

To continue the transient simulation, back up controlDict, set the start mode to continue from the latest saved time, and increase endTime. Do not confuse this with post-processing.

```
grep -nE 'startFrom|startTime|stopAt|endTime|deltaT|writeControl|writeInterval|maxCo' system/controlDict
cp system/controlDict system/controlDict.before-extension
```

Then edit only the intended controls, for example set `startFrom latestTime` and choose an endTime beyond 2e-5 s. Check write control and time-step behaviour. Run with a new log name. The exact longer duration must be chosen from the physical phenomena and the force/probe histories, not from an arbitrary fixed rule.

```
set -o pipefail
foamRun 2>&1 | tee logs/extended-run-01.log
```

The chord-flow time is about 2.30 ms. This is a useful scale, not a universal stopping time. Shock adjustment, viscous development, domain reflections and unsteady behaviour can require longer. Examine the actual time histories.

28. Validation programme: what must happen before comparing with the paper

Validation is the process of asking whether the numerical predictions agree with independent physical/reference evidence for the intended problem. Before paper comparison, the numerical solution should also be verified for sensitivity to the mesh, time step and domain.

28.1 Mesh sensitivity

- Repair the high non-orthogonality, especially near the cusp. Consider a different block layout, smoother projected edges or better clustering.
- Create at least coarse, medium and fine meshes while holding physical setup and domain fixed.
- Compare wall Cp, pressure distribution, shock position, wall shear and Cd/Cl/Cm at matched physical times or after a consistent averaging interval.
- Compare near-wall spacing and cell count across upper/lower surfaces; do not refine only the far field.

28.2 Domain sensitivity

The current bounding box is x=-1.25..4 m and z=-2..2 m. The paper reports a much larger far-field region, with an 80-chord-diameter final domain. The current outer boundary may influence waves and pressure disturbances. Enlarge the domain and compare the field and wall loads rather than assuming the smaller domain is equivalent.

28.3 Time-step sensitivity

Repeat the same physical problem with a smaller time step (for example half the current deltaT) while controlling the same physical end time. Compare force histories and local pressure/Mach profiles. Keep the mesh and physical settings fixed during this test.

28.4 Boundary and physical-model review

- Verify the pressure reference convention and derive density/Re from absolute pressure consistently.
- Check whether the current inlet/outlet patch conditions are appropriate for the supersonic outer-flow layout. The paper used Fluent pressure-far-field conditions; the current simple fixedValue/zeroGradient arrangement is an adaptation.
- Verify gamma, Cp/Cv, molecular weight, viscosity law and Prandtl/thermal conductivity treatment against the paper.
- Check alpha=0° velocity vector and later rotate components for nonzero angle of attack, rather than changing geometry orientation casually.
- Verify force coefficient directions, reference area and pitch axis for every angle of attack.

28.5 Compare like with like

Only compare the same Mach number, Reynolds number, angle of attack, geometry, coefficient definition, reference axes and statistical time window. If the paper reports time-averaged values, compare against a time-averaged OpenFOAM result after it reaches repeatable behaviour, not a startup snapshot.

29. A practical sequence for the next working sessions

Session	Goal	Concrete output
1	Archive current milestone and inspect all dictionaries	Baseline case copy, file hashes/notes, final logs.
2	Resolve geometry and mesh quality around the cusp	Improved mesh plus checkMesh comparison.
3	Confirm pressure convention, Reynolds number and thermo properties	Documented freestream calculation and corrected reference state.
4	Extend run and monitor Courant/residuals/field bounds	Longer transient solution with traceable log.
5	Force coefficient and dimensional-force histories	Cd/Cl/Cm and pressure/viscous loads versus time.
6	Point probes and selected field profiles	Probe histories and repeatable line plots.
7	Mesh/time-step/domain sensitivity	Comparison tables and error/sensitivity measures.
8	Construct final Cp and Cf distributions	Upper/lower surface distributions with correct sign/reference conventions.
9	Compare with paper at matched operating conditions	Comparison plots/tables plus differences and caveats.
10	Prepare an engineering report and reproducibility package	README, dictionaries, scripts, logs, plots, assumptions and limitations.

30. Reproducible engineering record

For each session, write down enough information that another person can repeat the same analysis and understand why the choice was made.

```
Date/time:
Case path and OpenFOAM version:
Question / purpose:
Saved time(s) analysed:
Files changed (and backups):
Commands run:
Output files created:
Observed values and units:
Interpretation (what is observation vs explanation?):
Limitations / uncertainty:
Next action:
```

A useful short note is not a transcript. It records the decision, evidence and unresolved uncertainty. Keep scripts and plots with descriptive names. Preserve a baseline case copy before major mesh or physics changes.

31. Troubleshooting reference

Symptom	Likely cause / check	Response
`field not found`	Selected time does not contain the field, or derived field has not been written.	List time files; run the correct function; select the intended time.
Line profile nearly uniform	Sampling line misses the disturbance or is at wrong y/z.	Check coordinates against mesh cell centres and geometry; inspect header and endpoints.
NaNs from point probes	Probe points outside fluid cells, inside solid or interpolation failed.	Use a cell locator; report valid/invalid counts; inspect failed locations.
`numpy` import error	Different Python interpreter/venv than the one where packages were installed.	Check `which python`; activate the intended venv; test imports.
Unexpected Mach extrema	Local state, boundary issue, or numerical/mesh feature.	Map cell centres and compare p/T/U fields in the same region.
Spiky vorticity or wall shear	Gradient sensitivity, high non-orthogonality or insufficient near-wall resolution.	Map the peak; inspect mesh; refine and compare.
Surface export lacks face values	Using point-only surface output or reading wrong VTK array association.	Use the `foamToVTK` patch file; inspect cell data separately from point data.
Force sign seems wrong	Drag/lift directions or patch normals not checked.	Inspect force vectors, area vectors, axes and a known symmetric case.
Force coefficient has surprising scale	Wrong rhoInf, UInf, Aref, lRef, pRef or span.	Recompute q∞ and q∞Aref; confirm units and geometry convention.

Symptom	Likely cause / check	Response
Mesh says OK but result unstable	The default checks passed but quality metrics remain poor.	Review detailed checkMesh warnings; perform mesh sensitivity.
Need more simulated time	ControlDict still stops at old endTime.	Back up; set startFrom latestTime; increase endTime; run a new solver log.

32. What to say about the current milestone

Defensible technical summary

A custom cusped-leading-edge airfoil geometry was prepared from coordinate data and checked as an STL. A structured multi-block mesh was adapted from the OpenFOAM Foundation 13 NACA0012 tutorial, refined to 24,000 hexahedral cells and configured as a two-dimensional X-Z case. The flow setup was adapted from the Foundation 13 shockFluid diffuserIntake tutorial to a laminar, compressible ideal-gas model with Sutherland viscosity. A short transient run completed to 2×10^-5 s. Post-processing has produced Mach/speed/vorticity fields, line profiles, wall-patch data, near-wall probes, wall-shear analysis, geometric closure checks and pressure/viscous force integration. Python integration agrees closely with the Foundation v13 forcesCompressible and forceCoeffsCompressible results for the same snapshot. The result remains preliminary because the mesh has maximum non-orthogonality around 87.27°, the domain is smaller than the paper's final far field, the simulated interval is only about 0.87% of one chord convective time, and pressure convention/boundary-condition equivalence plus mesh/time-step/domain sensitivity remain to be resolved.

33. Key terms glossary

Term	Meaning in this project
Cell	A finite-volume control volume in which OpenFOAM stores internal fields.
Face	A polygon separating cells or belonging to a boundary patch.
Patch	A named group of boundary faces such as `aerofoil`, `inlet` or `outlet`.
STL	Triangulated surface geometry used for projection/meshing and visual reference.
Non-orthogonality	Angle-related measure of how far a face normal departs from the line joining adjacent cell centres; high values affect gradient/flux accuracy.
Skewness	Measure of face-centre displacement from ideal geometry; high values can degrade interpolation.
Mach number	Local flow speed divided by local speed of sound.
Vorticity	Curl of velocity, ∇×U; a measure of local rotation/shear.
Wall shear stress	Tangential viscous stress acting at the solid boundary.
Cp	Dimensionless static-pressure coefficient relative to freestream dynamic pressure.
Cf	Dimensionless wall-shear coefficient; specify magnitude or signed tangential convention.
Aref	Reference area used to nondimensionalise forces; not the same as wetted area.
CofR	Centre of rotation about which moments are reported.
Verification	Checking numerical implementation and consistency (e.g. independent force integration).
Validation	Checking whether the model predicts the intended real/reference flow accurately.

34. References and provenance

This guide synthesises the earlier 14-page "From Foam Run to CFD Evidence" post-processing guide, the 16-page "Case Management & Post-Processing Field Guide", the project progress record, and the subsequent wall/force analysis described in the work history. Case-specific numerical values are tied to the observed logs and extracted data described in those records.

OpenFOAM Foundation v13 documentation:

- Chapter 7, Post-processing: https://doc.cfd.direct/openfoam/user-guide-v13/postprocessing
- Section 7.3, Post-processing functionality (field calculation, forces, graph sampling, probes, surface fields): https://doc.cfd.direct/openfoam/user-guide-v13/post-processing-functionality
- OpenFOAM v13 User Guide index (case structure, meshing, boundary conditions, physical properties): https://doc.cfd.direct/openfoam/user-guide-v13/index

Research paper: Akram, S. and Hasan, N. (2024), “The dynamics of supersonic flow past a new cusped leading edge airfoil,” Progress in Computational Fluid Dynamics, 24(1), 1-28. DOI: 10.1504/PCFD.2024.135636. https://doi.org/10.1504/PCFD.2024.135636

The official v13 guide describes `foamPostProcess`, derived fields, compressible force functions, graph sampling, probes and surface-field operations. The research paper description confirms the laminar, viscous, compressible and unsteady nature of the study, and the Mach/Reynolds-number/angle-of-attack ranges. Exact quantitative comparison requires the full paper and the same modelling definitions.